

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

Task -1: Create Project Structure as follows:

OSSP/

```
|
|
├── .git/           # Git repository (created automatically)
├── .gitignore      # Files/folders Git should ignore
├── README.md       # Project description and instructions
├── LICENSE         # Project license (optional)
├── Makefile        # Build instructions
|
|
├── src/            # Source code
|   ├── main.c
|   ├── shell.c
|   ├── parser.c
|   ├── executor.c
|   └── builtin.c
|
|
├── include/        # Header files
|   ├── shell.h
|   ├── parser.h
|   ├── executor.h
|   └── builtin.h
|
|
├── obj/            # Object files (.o) (generated)
|
|
├── bin/            # Executable file (generated)
|   └── my_shell
|
|
├── docs/           # Documentation
|   ├── design.md
|   └── report.pdf
|
|
├── tests/          # Test programs
|   ├── test_parser.c
|   └── test_executor.c
|
|
├── scripts/        # Helper scripts
|   └── run.sh
|
|
```

```
└── assets/           # Images/screenshots
    └── architecture.png
```

Note: For creating this structure use `mkdir` and `touch` commands.

```
speed@DESKTOP-P4G181S:~$ # 1. Create main directory and enter it
mkdir -p OSSP && cd OSSP

# 2. Initialize Git repository
git init

# 3. Create all subdirectories
mkdir -p src include obj bin docs tests scripts assets

# 4. Create root-level files
touch .gitignore README.md LICENSE Makefile

# 5. Create source files in src/
touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c

# 6. Create header files in include/
touch include/shell.h include/parser.h include/executor.h include/builtin.h

# 7. Create documentation, test, script, and asset files
touch docs/design.md docs/report.pdf
touch tests/test_parser.c tests/test_executor.c
touch scripts/run.sh
touch assets/architecture.png

# 8. (Optional) Verify the directory tree structure
tree -a
Reinitialized existing Git repository in /home/speed/OSSP/.git/
```

```
.
├── .git
│   ├── HEAD
│   ├── config
│   ├── description
│   ├── hooks
│   │   ├── applypatch-msg.sample
│   │   ├── commit-msg.sample
│   │   ├── fsmonitor-watchman.sample
│   │   ├── post-update.sample
│   │   ├── pre-applypatch.sample
│   │   ├── pre-commit.sample
│   │   ├── pre-merge-commit.sample
│   │   ├── pre-push.sample
│   │   ├── pre-rebase.sample
│   │   ├── pre-receive.sample
│   │   ├── prepare-commit-msg.sample
│   │   ├── push-to-checkout.sample
│   │   ├── sendemail-validate.sample
│   │   └── update.sample
│   ├── info
│   │   └── exclude
│   ├── objects
│   │   ├── info
│   │   └── pack
│   └── refs
│       ├── heads
│       └── tags
├── .gitignore
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── shell
├── shell.c
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c
```

- 
1. **To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**

Task-1: Using a manual command understand the fork() and exec() system calls

fork(2)	System Calls Manual	fork(2)
NAME	fork - create a child process	
LIBRARY	Standard C library (<code>libc</code>)	
SYNOPSIS	<pre>#include <unistd.h> pid_t fork(void);</pre>	
DESCRIPTION	<code>fork()</code> creates a new process by duplicating the calling process. The new process is referred to as the <i>child</i> process. The calling process is referred to as the <i>parent</i> process. The child process and the parent process run in separate memory spaces. At the time of <code>fork()</code> both memory spaces have the same content. Memory writes, file mappings (<code>mmap(2)</code>), and unmappings (<code>munmap(2)</code>) performed by one of the processes do not affect the other. The child process is an exact duplicate of the parent process except for the following points: - The child has its own unique process ID, and this PID does not match the ID of any existing process group (<code>setpgid(2)</code>) or session. - The child's parent process ID is the same as the parent's process ID. - The child does not inherit its parent's memory locks (<code>mlock(2)</code>, <code>mlockall(2)</code>). - Process resource utilizations (<code>getrusage(2)</code>) and CPU time counters (<code>times(2)</code>) are reset to zero in the child. - The child's set of pending signals is initially empty (<code>sigpending(2)</code>). - The child does not inherit semaphore adjustments from its parent (<code>semop(2)</code>). - The child does not inherit process-associated record locks from its parent (<code>flock(2)</code>). (On the other hand, it does inherit <code>flock(2)</code> open file description locks and <code>flock(2)</code> locks from its parent.) - The child does not inherit timers from its parent (<code>setitimer(2)</code>, <code>alarm(2)</code>, <code>timer_create(2)</code>). - The child does not inherit outstanding asynchronous I/O operations from its parent (<code>aio_read(3)</code>, <code>aio_write(3)</code>), nor does it inherit any asynchronous I/O contexts from its parent (see <code>io_setup(2)</code>). The process attributes in the preceding list are all specified in POSIX.1. The parent and child also differ with respect to the following Linux-specific process attributes: - The child does not inherit directory change notifications (<code>inotify</code>) from its parent (see the description of <code>F_NOTIFY</code> in <code>fcntl(2)</code>). - The <code>prctl(2)</code> <code>PR_SET_PODATHSIG</code> setting is reset so that the child does not receive a signal when its parent terminates. - The default timer slack value is set to the parent's current timer slack value. See the description of <code>PR_SET_TIMER_SLACK</code> in <code>prctl(2)</code>. - Memory mappings that have been marked with the <code>madvise(2)</code> <code>MADV_DONTRECK</code> flag are not inherited across a <code>fork()</code>. - Memory in address ranges that have been marked with the <code>madvise(2)</code> <code>MADV_WIPEONFORK</code> flag is zeroed in the child after a <code>fork()</code>. (The <code>MADV_WIPEONFORK</code> setting remains in place for those address ranges in the child.) - The termination signal of the child is always <code>SIGCHLD</code> (see <code>clone(2)</code>). - The port access permission bits set by <code>lperm(2)</code> are not inherited by the child; the child must turn on any bits that it requires using <code>lperm(2)</code>. Note the following further points: - The child process is created with a single thread—the one that called <code>fork()</code>. The entire virtual address space of the parent is replicated in the child, including the states of mutexes, condition variables, and other pthreads objects; the use of <code>pthread_atfork(3)</code> may be helpful for dealing with problems that this can cause. - After a <code>fork()</code> in a multithreaded program, the child can safely call only <i>async-signal-safe</i> functions (see <code>signal-safety(7)</code>) until such time as it calls <code>execve(2)</code>. - The child inherits copies of the parent's set of open file descriptors. Each file descriptor in the child refers to the same open file description (see <code>open(2)</code>) as the corresponding file descriptor in the parent. This means that the two file descriptors share open file status flags, file offset, and signal-driven I/O attributes (see the description of <code>F_SETOWN</code> and <code>F_SETSIG</code> in <code>fcntl(2)</code>). - The child inherits copies of the parent's set of open message queue descriptors (see <code>mq_overview(7)</code>). Each file descriptor in the child refers to the same open message queue description as the corresponding file descriptor in the parent. This means that the two file descriptors share the same flags (<code>mq_flags</code>). - The child inherits copies of the parent's set of open directory streams (see <code>opendir(3)</code>). POSIX.1 says that the corresponding directory streams in the parent and child <i>may</i> share the directory stream positioning; on Linux/glibc they do not.	
RETURN VALUE	On success, the PID of the child process is returned in the parent, and 0 is returned in the child. On failure, -1 is returned in the parent, no child process is created, and <code>errno</code> is set to indicate the error.	
ERRORS	EAGAIN A system-imposed limit on the number of threads was encountered. There are a number of limits that may trigger this error: - the RLIMIT_NPROC soft resource limit (set via <code>setrlimit(2)</code>), which limits the number of processes and threads for a real user ID, was reached; - the kernel's system-wide limit on the number of processes and threads, <code>/proc/sys/kernel/threads-max</code>, was reached (see <code>proc(5)</code>); - the maximum number of PIDs, <code>/proc/sys/kernel/pid_max</code>, was reached (see <code>proc(5)</code>); or - the PID limit (<code>pids_max</code>) imposed by the cgroup "process number" (PIDs) controller was reached. EAGAIN The caller is operating under the SCHED_DEADLINE scheduling policy and does not have the <code>reset-on-fork</code> flag set. See <code>sched(7)</code>. ENOMEM <code>fork()</code> failed to allocate the necessary kernel structures because memory is tight. ENOMEM An attempt was made to create a child process in a PID namespace whose "init" process has terminated. See <code>pid-namespaces(7)</code>. EINVAL <code>fork()</code> is not supported on this platform (for example, hardware without a Memory-Management Unit). EBRESTARTNOINTR (since Linux 2.6.17) System call was interrupted by a signal and will be restarted. (This can be seen only during a trace.)	
VERSIONS	C library/kernel differences Since glibc 2.3.3, rather than invoking the kernel's <code>fork()</code> system call, the glibc <code>fork()</code> wrapper that is provided as part of the NPTL threading implementation invokes <code>clone(2)</code> with flags that provide the same effect as the traditional system call. (A call to <code>fork()</code> is equivalent to a call to <code>clone(2)</code> specifying flags as just <code>SIGCHLD</code>.) The glibc wrapper invokes any fork handlers that have been established using <code>pthread_atfork(3)</code>. Async-signal safety <code>_fork(3)</code> is an <i>async-signal safe</i> variant of <code>fork(2)</code>.	
STANDARDS	POSIX.1-2004.	
HISTORY	4.3BSD, SysV, POSIX.1-1988.	
NOTES	Under Linux, <code>fork()</code> is implemented using copy-on-write pages, so the only penalty that it incurs is the time and memory required to duplicate the parent's page tables, and to create a unique task structure for the child.	
EXAMPLES	See <code>pipe(2)</code> and <code>wait(2)</code> for more examples.	
	<pre>#include <signal.h> #include <stdlib.h> #include <stdio.h> #include <unistd.h> #include <sys/types.h> int main(void) { pid_t pid; if (signal(SIGCHLD, SIG_IGN) == SIG_ERR) { perror("signal"); } }</pre>	Manual page fork(2) line 1/178 98% (press h for help or q to quit)

Task-2: Write a C program to create a new process and accepting the user commands using `fork()` and `exec()` system calls.

Example:

```
#include <stdio.h>
// ...
```

```
#include <unistd.h>

int main() {
    pid_t pid;
    pid = fork(); // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1) {
        // Display prompt
        printf("my_cmd_shell> ");
        fflush(stdout);

        // Read command from user
        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\nExiting...\n");
            break;
        }

        // Remove trailing newline character
        input[strcspn(input, "\n")] = '\0';

        // Skip empty input
        if (strlen(input) == 0) {
            continue;
        }

        // Handle exit condition
        if (strcmp(input, "exit") == 0) {
            printf("Exiting shell.\n");
            break;
        }

        // Parse command into arguments (tokenize by space)
        int i = 0;
        char *token = strtok(input, " ");
        while (token != NULL && i < MAX_ARGS - 1) {
            args[i++] = token;
            token = strtok(NULL, " ");
        }
        args[i] = NULL; // Array must be NULL-terminated for execvp

        // Create child process
        pid_t pid = fork();

        if (pid < 0) {
            perror("Fork failed");
        }
        else if (pid == 0) {
            // Child Process: Execute the command
            printf("[Child PID: %d] Executing: %s\n", getpid(), args[0]);
            if (execvp(args[0], args) < 0) {
                perror("Command execution failed");
                exit(EXIT_FAILURE);
            }
        }
        else {
            // Parent Process: Wait for child process to terminate
            int status;
            waitpid(pid, &status, 0);
            printf(" [Parent PID: %d] Child process %d finished.\n", getpid(), pid);
        }
    }

    return 0;
}

```

```
Ubuntu X + v
gcc process_exec.c -o process_exec

# Run the program
./process_exec
my_cmd_shell> ls -l
[Child PID: 845] Executing: ls
total 192
-rw-r--r-- 1 speed speed 78 Aug 5 09:37 File1.txt
-rw-r--r-- 1 speed speed 78 Aug 5 09:39 File2.txt
-rwxr-xr-x 1 speed speed 15952 Jul 25 03:54 Hello
-rw-r--r-- 1 speed speed 80 Jul 25 03:41 JOE
drwxr-xr-x 11 speed speed 4096 Aug 4 07:09 OSSP
-rw-r--r-- 1 speed docker 387 Jul 25 03:48 auto-execute.md
-rwxr-xr-x 1 speed speed 16176 Aug 5 09:39 filecopy
-rw-r--r-- 1 speed speed 563 Aug 5 09:39 filecopy.c
-rwxr-xr-x 1 speed speed 16088 Aug 4 06:54 fork_example
-rw-r--r-- 1 speed speed 444 Aug 4 06:54 fork_example.c
-rw-r--r-- 1 speed speed 80 Jul 25 03:54 hi.c
-rwxr-xr-x 1 speed speed 16264 Aug 4 06:36 process
-rw-r--r-- 1 speed speed 702 Aug 4 06:36 process.c
-rwxr-xr-x 1 speed speed 16608 Aug 14 09:51 process_exec
-rw-r--r-- 1 speed speed 1851 Aug 14 09:49 process_exec.c
drwxr-xr-x 3 speed docker 4096 Jul 25 03:56 projects
-rwxr-xr-x 1 speed speed 16136 Aug 14 09:42 read_demo
-rw-r--r-- 1 speed speed 372 Aug 14 09:42 read_demo.c
-rwxr-xr-x 1 speed speed 16216 Aug 5 09:33 shell
-rw-r--r-- 1 speed speed 419 Aug 5 09:33 shell.c
drwxr-xr-x 2 speed speed 4096 Aug 11 06:51 simple_shell
-rwxr-xr-x 1 speed speed 16304 Aug 14 09:33 task1
-rw-r--r-- 1 speed speed 1024 Aug 14 09:33 task1.c
[Parent PID: 841] Child process 845 finished.
my_cmd_shell> pwd
[Child PID: 846] Executing: pwd
/home/speed
[Parent PID: 841] Child process 846 finished.
my_cmd_shell> date
[Child PID: 847] Executing: date
Fri Aug 14 09:51:50 UTC 2026
[Parent PID: 841] Child process 847 finished.
my_cmd_shell> whoami
[Child PID: 848] Executing: whoami
speed
[Parent PID: 841] Child process 848 finished.
my_cmd_shell> exit
Exiting shell.
speed@DESKTOP-P4G181S: $
```